MORNING · BRIEF

每日晨间简报

星期二 · 2026年10月13日 *Vol. 2026 · No. 286*

凡是我不能创造的,我就不理解。

“What I cannot create, I do not understand.”

—— 理查德·费曼 · 1988 年他去世时留在加州理工学院办公室黑板上的一句话

今日一首歌

贝加尔湖畔 — 李健

中文民谣,吉他和弦乐铺底,节奏很慢,李健的嗓音干净平稳。适合晚上读书或者写完代码收尾时听。

今天先做 CodeCrafters HTTP 服务器 at4(附 ia4),再做 chat-cli 第 3 天:多轮对话。

一、今日三条新闻

I 面壁智能开源 MiniCPM-V 4.7:350 亿参数的模型装进手机

面壁智能和 OpenBMB 社区 10月13日发布并开源 MiniCPM-V 4.7,有 35B-A3B 和 1B 两个版本,是这个系列第一个“混合专家”(MoE)模型。Hugging Face 模型页称,它在 iPhone 17 Pro Max 上只用 3 GB 内存就能每秒生成 13 个以上的词元,许可证是 Apache-2.0。(AIHOT)

为什么值得看 ▸ “总共 350 亿、每次只用约 30 亿”就是混合专家,今天英语精读讲它。

II Linus Torvalds 用 Claude Code 写内核补丁,读缓存快了约 3 倍

Linuxiac 10月12日报道,Torvalds 10月10日提交了一组 8 个补丁的实验,改进 Linux 从页缓存读数据的方式。他写明代码、测试和说明初稿是“在我指导下”由 Claude Code 完成的;在他笔记本上的一项测试里,8 个线程读同一个缓存文件,每秒读取次数从约 1000 万升到 3000 万以上。这只是实验,没有合进主线。

为什么值得看 ▸ 方向他定、测量他自己做——用 AI 写代码,判断还得是自己的。

III 黑客让法国约 400 家超市的冷柜一夜升温

The Week 10月13日报道,10月12日夜里,家乐福、Monoprix、Intermarché 等连锁的约 400 家门店冷柜被远程调热,据报道最高到 50°C,多处起火、食品报废。报道称攻击者利用的是丹佛斯(Danfoss)远程监控软件所在的旧设备,这些设备已不再收到安全更新;巴黎检察院网络犯罪部门 13 日立案调查。

为什么值得看 ▸ 不再更新的联网设备会造成真实损失;你写的服务器也连着网络。

所属方向: I 科技与 AI · II 编程与开发 · III 国内外要闻

GITHUB 今日项目 **tobi / disktree**

用 Rust 写的磁盘占用可视化工具:把每个文件夹画成按大小切分的“方块地图”,看清硬盘被什么占满,标记后统一清理。

Rust · 约 0.29 万星 · 创建于 2026年9月24日 · MIT 许可证 · 有发布版和 AUR 包,README 没写稳定程度

§ 有什么用

- 颜色区分代码、缓存、工具链、媒体等类型,斜线标出“可回收”的空间(缓存、构建产物等)。
- 删除前先标记、再复查;默认移到回收站,永久删除要再确认一次。

§ 为什么火

- 创建不到三周约 2,900 星,是这段时间新建的 Rust 仓库里少见的非“craft 系列”项目。
- Linux、macOS、Windows 都有原生图形界面;复查页还能一键生成给编程智能体的清理提示。

§ 怎么用

```
disktree              # 扫描主目录
disktree ~/src        # 或任意目录;从源码装用 make install
```

§ 要注意的

需要 Rust 1.97 以上;macOS 的发布版没签名,要按 README 手动解除隔离。以上来自 README 和 GitHub 统计,未实际安装。

§ 和你的关系

它把扫描逻辑放在不带界面的 `disktree-core` 里,删除规则每条都配测试——和 chat-cli 第 6 天“拆模块、写测试”是同一个思路。

二、今日任务

标“必做”的是项目(chat-cli 第 3 天)、今日练习(今天是 CodeCrafters)、隔天重写和填写记录;其余选做,做到哪算哪。

按你 12 日备注的要求,CodeCrafters 排在必做最前面,下一关 ia4 也放进来了。

打卡 · 复利

从 2026年10月10日 起算 · 截至昨天

×1.089 总累计

本月(10月):×**1.089**

本月 1.1 有 **2** 天,0.9 有 **1** 天

往月:—

已持续 **3** 天(10月10日起)

连续 1.1:第 **2** 天 · 状态:**未断**

2026年10月

一	二	三	四	五	六	日
			1	2	3	4
5	6	7	8	9	10 0.9	11 1.1
12 1.1	13 ?	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

黑底 = 1.1 · 虚线框 = 0.9 · 点线框 = 今天 · 灰 = 起算之前 · 细边空白 = 以后

☐ 今天:项目、CSES 或 CodeCrafters 有进展(完成或部分完成),或做了隔天重写,就是 1.1——总累计变成 **1.198**;一项都没动是 0.9——变成 **0.980**。

回你的备注

12 日那天,项目、CSES、CodeCrafters、跑步、读书都有,是这几天最完整的一天。你提的三件事:

先做 CodeCrafters、讲细一点、把下一关也排上。照办:今天的练习就是 HTTP 服务器 at4,ia4 接在同一张卡片后半;CSES 1092 挪到明天。

1072 的式子。k^2 个格子里任挑两个,是 $k^2(k^2-1)/2$ 种。两只马能互相攻击时,它们一定在某个 2×3 或 3×2 小长方形的两个对角上,每个这样的小长方形里正好有 2 对。k×k 棋盘里 2×3 的有 (k−1)(k−2) 个,3×2 的也一样多,合起来 2×2×(k−1)(k−2),就是 4(k−1)(k−2)。今天重写前先在 3×3 上数出 8 对,再对一下这个式子。

“光有思路,不知道怎么写 Rust”。你自己已经看到了区别:chat-cli 写得出来,是因为指南把每一步拆到了一两行代码的粒度。所以 at4 第 1 步是先用中文给每一行写注释,别跳过。ni6 借 AI 过关不要紧,要紧的是之后自己从空白再写一遍——这就是“隔天重写”的用处。

从这里开始 **CodeCrafters at4 第 1 步:**打开 `src/main.rs`,先不改,用中文给每一行(包括被注释掉的)写一句它在做什么。

☐ CodeCrafters HTTP 服务器 · at4 Bind to a port(附 ia4)

必做

关卡内容来自 GitHub 上的公开关卡说明,这次没有读到登录后的页面。第一次进这门课时,先按网页指引开始课程、选 Rust、把仓库克隆到本地。

§ 思路

HTTP 跑在 TCP 之上,所以服务器要先做两件分开的事:**占住端口**(bind,一次)和**一个一个接受连接**(incoming,循环)。at4 只检查“能连上 4221”,不发也不收任何数据;ia4 在同一个循环里,对每个连接原样写回一串固定的字节——难点只在于这串字节要以一个空行结尾。

at4 · Bind to a port

at4-1 **读懂起始代码。**

先不改。用中文给每一行写一句话:哪一行在占住端口?哪一行在等下一个连接?match 的 Ok 和 Err 各在什么时候执行?

> **怎么确认 ▸** 能用自己的话说出 bind、incoming()、Ok、Err 各在干什么;说不出的那行打问号,做完第 3、4 步再回来看。

at4-2 **先看“没人在听”的样子。**

不改代码,一个终端运行 `./your_program.sh`,另开一个终端发请求。

> **怎么确认 ▸** curl 报连接被拒绝,因为还没有程序在 4221 上监听:

```
$ curl -v http://localhost:4221
curl: (7) Failed to connect to localhost port 4221 … Connection refused
```

at4-3 **占住端口。**

在 127.0.0.1:4221 上建一个 TcpListener,下面临时加一行打印 “listening on 4221”。

> **怎么确认 ▸** 运行后看到这行字、没有 panic,程序随即结束(还没有循环)。看到 Address already in use,说明上一次的程序还开着。

at4-4 **一个接一个地接受连接。**

加上 for 循环和 match:Ok 分支打印 “accepted new connection”,Err 分支打印错误、不退出。

> **怎么确认 ▸** 服务器停在那里不退出;另开终端 curl,服务器打印一行 accepted new connection,curl 显示:

```
* Connected to localhost (127.0.0.1) port 4221
curl: (52) Empty reply from server
```

at4-5 **跑测试并提交。**

`codecrafters test`,通过后 git add、commit、push。

> **怎么确认 ▸** 网页显示 at4 通过,进度进到 ia4。

ia4 · Respond with 200(at4 通过后接着做)

ia4-1 **先在纸上写出要发的字节。**

抄一遍 `HTTP/1.1 200 OK\r\n\r\n`,在两个 \r\n 下面各写一句:它结束的是什么?

> **怎么确认 ▸** 第一个结束状态行;第二个是空行,表示“头部到此为止”。少了它,curl 会一直等。

ia4-2 **让 Ok 分支能写数据。**

把 `Ok(_stream)` 改成 `Ok(mut stream)`;文件开头引入 `std::io::Write`。

> **怎么确认 ▸** `cargo build` 通过(可能有“Write 没用上”的警告,下一步就没了)。

ia4-3 **把响应写回去。**

在 Ok 分支里用 `write_all` 写入第 1 步那串字节(它要的是字节,不是 &str)。

> **怎么确认 ▸** curl 不再报 Empty reply,能看到以 < 开头的状态行:

```
$ curl -v http://localhost:4221
> GET / HTTP/1.1
> Host: localhost:4221
…
< HTTP/1.1 200 OK
```

ia4-4 **自己当一回客户端。**

有 nc 的话运行:`printf 'GET / HTTP/1.1\r\n\r\n' | nc localhost 4221`

> **怎么确认 ▸** 终端打印 `HTTP/1.1 200 OK` 和一个空行。程序没看请求内容也照样回 200,这正是这一关的要求。

ia4-5 **跑测试并提交。**

`codecrafters test`,通过后提交。

> **怎么确认** ▸ ia4 通过、at4 仍通过,进度进到 ih0。

§ 相关知识

- **bind 和 incoming 都返回 Result。** `TcpListener::bind("127.0.0.1:4221")` 可能失败(端口被占);`incoming()` 每次给的是 `Result<TcpStream>`,所以要 match 出 Ok 和 Err——今天的 Rust 知识点正好讲这个。
- **write_all 来自 Write trait。** 不 `use std::io::Write;` 就找不到这个方法;写进去的是 `&[u8]`,用 `.as_bytes()` 或字节串 `b"…"`。
- **CRLF。** HTTP 的每一行以 `\r\n` 结尾,头部结束再多一个 `\r\n`;今天这串响应一共 19 个字节。
- **连接什么时候关。** Ok 分支结束时 stream 被丢弃,连接就关了——at4 里 curl 收到 Empty reply 就是因为这个。

明天:CSES 1092 Two Sets(CodeCrafters 停在你做到的那一关)。

☐ chat-cli 项目 · 第 3 天:多轮对话

必做

目标:程序变成一个可以连续聊天的循环,模型能记住前面说过的话。

今天最重要的认知:模型本身没有记忆。它“记得”上文,是因为你每次都把全部历史重新发过去。

§ 任务清单

- ☐ 把“发请求并返回回复”抽成一个函数,参数是消息列表
- ☐ 用 `Vec<Message>` 保存历史,第一条放 system 消息
- ☐ 写循环:打印提示符,读一行输入,发请求,打印回复
- ☐ 每轮把用户消息和模型回复都追加进历史
- ☐ 支持 `/exit` 退出、`/clear` 清空历史(保留 system 消息)
- ☐ 空输入直接跳过,不发请求

§ 提示

- 读输入:`std::io::stdin().read_line(&mut input)`,读到的字符串带换行符,要 `.trim()`。
- `print!("> ")` 不会立刻显示,后面要跟 `std::io::stdout().flush()`,需要 `use std::io::Write;`。
- `read_line` 返回 `Ok(0)` 表示输入结束,此时也应该退出循环。
- 函数参数用 `&[Message]`,比 `&Vec<Message>` 更通用。
- role 一共三种:`system`、`user`、`assistant`。

§ 常见坑

- 请求失败时,刚追加的用户消息还留在历史里,下一轮会连着发两条 user 消息。失败时记得把它弹出去。
- 循环里出错不要用 `?` 直接退出程序,打印错误后继续下一轮。

§ 实验

每轮打印 `usage.prompt_tokens`。连续聊几轮,你会看到这个数字一轮比一轮大,这就是“每次重发全部历史”的代价。

完成标准 先告诉模型你的名字,隔两轮再问“我叫什么”,它能答对;`/clear` 之后再问,它答不出来。

照你的 7 天指南 第 3 天写。做完后在记录页把项目标为完成,简报才会进到第 4 天;卡住时把做到第几条和报错填进项目那一栏。

☐ 隔天重写 · CSES 1072 Two Knights

必做

今天要重写的:CSES 1072(12 日做完)。12 日的 CodeCrafters 关卡 ni6 属于已经停掉的 shell 课程,不重写。

做法:合上昨天的代码,新建文件从空白开始;先用中文注释写出每一步,再一行行翻成 Rust;用题目样例验证,输出一字不差;写不出来的地方记进记录页“重写时哪里又写不出来了”。

§ 对照清单 · 要自己写出来的关键处

- ☐ 读入 n,结果用哪种整数类型才不溢出
- ☐ 全部放法数(k^2 个格子挑两个)
- ☐ 互相攻击的放法数(回信里那个“2×3 小长方形”的数法)
- ☐ k 从 1 到 n 每行一个结果,拼进一个 String 最后一次输出

重写完在记录页勾上“重写了前一天的 CSES 题”。

☐ Rust 知识点 · 第 3 讲:match 与 if let

选做

一句话:程序经常要“看一个值长什么样,再决定做什么”。`match` 把所有可能一条条列出来,每种情况对应一段代码,编译器还会检查有没有漏掉哪种情况;`if let` 是只关心其中一种情况时的简写。

第一部分 · 书里讲的

- **match 像分拣台。**值从上往下依次和每个“模式”比,第一个对上的分支被执行,后面的不再看。模式可以是一个具体的值(`'"'`)、几个值之一(`' ' | '\t'`)、一个范围(`'0'..='9'`),或者一个名字(把值绑定到这个名字上)。
- **必须穷尽。**所有可能的值都要有分支接住,漏了就编不过。不想一个个列时,用 `_` 收下“其余所有情况”。
- **match 是表达式,有值。**可以写 `let s = match … { … };`,每个分支的值类型必须一样。
- **和 Option 是一对。**`Option` 只有 `Some(x)` 和 `None` 两种样子,match 能把 x 取出来,同时逼你想清楚“没有值时怎么办”。
- **if let:只关心一种情况。**`if let Some(x) = 值 { … }` 等于只写了 Some 分支、其余什么也不做的 match。`while let` 同理:只要还对得上就一直循环。

例 1(能编译):几种模式写法。输出依次是 字母、数字、空白、引号、其他。

```rust
fn kind(c: char) -> &'static str {
    match c {
        ' ' | '\t' => "空白",
        '0'..='9' => "数字",
        '\'' | '"' => "引号",
        c if c.is_alphabetic() => "字母",
        _ => "其他",
    }
}

fn main() {
    for c in "a1 '?".chars() {
        println!("{c:?} -> {}", kind(c));
    }
}
```

例 2(这段编不过):以为 `n % 2` 只有 0 和 1。编译器报 `E0004: non-exhaustive patterns`,说 `i32::MIN..=-1` 和 `2..=i32::MAX` 没被覆盖——i32 的余数可能是负数(−7 % 2 等于 −1)。改法:最后一个分支换成 `_ => "奇数"`。

```rust
fn main() {
    let n: i32 = 7;
    let s = match n % 2 {
        0 => "偶数",
        1 => "奇数",
    };
    println!("{s}");
}
```

例 3(能编译): `while let` 加 `if let`,遇到反斜杠时多取一个字符。输出:普通 'a'、转义 ' '、普通 'b'。

```
fn main() {
    let mut it = "a\\ b".chars();
    while let Some(c) = it.next() {
        if c == '\\' {
            if let Some(next) = it.next() {
                println!("转义: {next:?}");
            }
            continue;
        }
        println!("普通: {c:?}");
    }
}
```

第二部分 · 书里没细讲,但你应该知道的

- **为什么要“穷尽”**。C 和 Java 的 switch 漏写一种情况不报错,程序悄悄什么都不做;Python 的 if/elif 也一样。Rust 把“漏情况”变成编译错误,好处在改代码时最明显:给枚举加一个新成员,所有没处理它的 match 都会被编译器一个个指出来。
- **守卫(guard)和分支顺序**。`c if c.is_alphabetic() =>` 里的 `if` 叫守卫:模式对上之后还要条件为真才进这个分支。分支从上往下比,更具体的写在前面,`_` 写在最后;写反了编译器会警告 unreachable pattern。
- **同时看几样东西:匹配元组**。`match (a, b) { … }` 一次看两个值,写状态机很好用,例 4 是经典的 FizzBuzz。
- **Option 的常用方法,很多时候不必写 match**。`unwrap_or(默认值)`、`map(函数)`、`ok()`(把 Result 变成 Option),还有 `let … else`:对不上立刻 return,对上了就把值拿出来接着用。例 5 演示这几种。
- **unwrap 什么时候可以用**。刷 CSES 时输入格式有保证,`parse().unwrap()` 没问题;服务器和聊天程序这种一直运行的程序,一次出错就 unwrap 会让整个程序退出——所以 at4 的起始代码对每个连接用 match 分开处理 Ok 和 Err。

例 4(能编译): 匹配元组。输出 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz。

```
fn say(x: u32) -> String {
    match (x % 3, x % 5) {
        (0, 0) => "FizzBuzz".to_string(),
        (0, _) => "Fizz".to_string(),
        (_, 0) => "Buzz".to_string(),
        _ => x.to_string(),
    }
}

fn main() {
    let v: Vec<String> = (1..=15).map(say).collect();
    println!("{}", v.join(" "));
}
```

例 5(能编译):不写 match 也能处理 Option。输出依次是 42、Some(84)、echo、(空行)。

```
fn first_word(line: &str) -> &str {
    let Some(w) = line.split_whitespace().next() else {
        return "(空行)";
    };
    w
}

fn main() {
    let n: Option<i32> = "42".parse().ok();
    println!("{}", n.unwrap_or(0));
    println!("{:?}", n.map(|x| x * 2));
    println!("{}", first_word("  echo hi"));
    println!("{}", first_word("   "));
}
```

§ 常见错误

- 把 `_ =>` 或 `c =>` 写在最前面,后面的分支全都到不了 → 把“什么都接”的分支挪到最后。
- 分支的值类型不一致,比如一个返回 `&str`、一个返回 `String`,报 `match arms have incompatible types` → 统一成一种,例如都用 `.to_string()`。

§ 小练习

不用 if/else,只用一个 match(可以带守卫),写 `fn sign(n: i64) -> &'static str`:0 返回“零”,大于 0 返回“正数”,小于 0 返回“负数”。

> **和今天任务的联系:**at4 里 `incoming()` 每次给一个 Result,`match stream { Ok(…) => …, Err(e) => … }` 两支缺一不可;ia4 把 `Ok(_stream)` 改成 `Ok(mut stream)`,说明模式还能顺手绑定名字、加 mut。chat-cli 第 3 天的输入循环也适合用 match:对 `input.trim()` 匹配 `"/exit"`、`"/clear"`、`""` 和 `_` 四种情况;`read_line` 的结果可以写 `Ok(0) =>` 来处理“输入结束”。

练习答案:`match n { 0 => "零", x if x > 0 => "正数", _ => "负数" }`,守卫分支要写在 _ 前面。 · 对应书里:6.2 match 控制流结构、6.3 if let 和 let else 简洁控制流。

☐ 英语 + AI 精读 · Many Experts, Few at Work

选做

§ Essay

¶1 A large language model is made of billions of numbers called **parameters**. In a normal **dense model**, every parameter is used to produce every **token**, the small piece of text the model writes next. More parameters usually mean a smarter model, but they also mean more memory and more work for every single token.

¶2 A **mixture of experts**, or MoE, changes this. Inside each layer, the model keeps many small networks called experts. A small part called the **router** looks at the current token and picks only a few experts to run. The other experts stay **idle**. So the model can hold a lot of knowledge, but it only pays for a small part of it at each step.

¶3 On October 13, the Chinese company ModelBest and the OpenBMB community released MiniCPM-V 4.7 as open source. Its large version is called 35B-A3B: about 35 billion parameters in total, but only about 3 billion **active** for each token. A review of the model files says each layer has 256 experts and uses 8 of them per token. The model card says the model can write more than 13 tokens per second on an iPhone 17 Pro Max with only 3 GB of memory.

¶4 Why does this matter? A model that runs **on-device** is cheaper to use, can answer without a network, and keeps your photos and messages on your own phone. But MoE is not free. All the experts must still be stored somewhere, and bringing the right ones into memory at the right time is hard. The model card lists a method called Lookahead Expert Scheduling for exactly this problem.

¶5 For a programmer, the idea feels familiar. A good program does not run every branch; it checks the input and runs only the branch it needs. A `match` expression in Rust works the same way.

§ 8 个关键词

parameter 参数(模型里的数)

dense model 稠密模型(每次全部参与)

token 词元(模型一次写出的一小段)

mixture of experts 混合专家

router 路由器(挑选专家的部分)

idle 空闲的

active 被激活的、正在工作的

on-device 在设备本地运行的

新闻原文:MiniCPM-V 4.7 35B-A3B 模型页(Hugging Face);“256 个专家、每次用 8 个”来自 OrcaRouter 对模型文件的整理;AIHOT 条目。

用中文写 3 句话总结:

§ 中文对照(先读完英文再看)· 专家很多,干活的很少

¶1 大语言模型由几十亿个叫作**参数**的数字组成。在普通的**稠密模型**里,模型每写出一个**词元**(接下来那一小段文字),所有参数都要参与计算。参数越多,模型通常越聪明,但每个词元要用的内存和计算量也越大。

¶2 **混合专家**(MoE)改变了这一点。模型的每一层里有许多叫作“专家”的小网络。一个叫**路由器**的小部件看一眼当前的词元,只挑几个专家来运行,其余专家**闲着**。这样模型能装下很多知识,每一步却只为其中一小部分付出计算。

¶3 10 月 13 日,中国公司面壁智能和 OpenBMB 社区开源了 MiniCPM-V 4.7。它的大版本叫 35B-A3B:总共约 350 亿参数,但每个词元只**激活**约 30 亿。一份对模型文件的整理称,每一层有 256 个专家,每个词元用其中 8 个。模型页说,它在 iPhone 17 Pro Max 上只用 3 GB 内存,每秒能写出 13 个以上的词元。

¶4 这为什么重要?**在设备本地**运行的模型用起来更便宜,没网也能回答,你的照片和消息也留在自己手机里。但 MoE 不是没有代价:所有专家仍要存放在某个地方,在恰当的时候把需要的那几个调进内存并不容易。模型页列出了一种叫 Lookahead Expert Scheduling(提前调度专家)的方法,专门解决这个问题。

¶5 对程序员来说,这个想法并不陌生。好的程序不会把每个分支都跑一遍,而是先看输入,只跑需要的那一支。Rust 的 `match` 表达式也是这样工作的。

先读完英文,再看关键词,最后对照中文。

☐ 英语原著阅读 · Handbook · 第 1 / 118 段:1.1 Programming languages 选做

今天读:Chapter 1 Introduction · 1.1 Programming languages(连同第 1 章开头的引言)· 书页 3–4(PDF 第 13–14 页)

§ Before you read

Q1 What two parts make up competitive programming, and why is a correct idea not enough?

Q2 Why does the author recommend C++, and why should you also know other languages?

Chapter 1 · Introduction

¶1 Competitive programming combines two topics: (1) the design of algorithms and (2) the implementation of algorithms.

¶2 The design of algorithms consists of problem solving and mathematical thinking. Skills for analyzing problems and solving them creatively are needed. An algorithm for solving a problem has to be both correct and efficient, and the core of the problem is often about inventing an efficient algorithm.

¶3 Theoretical knowledge of algorithms is important to competitive programmers. Typically, a solution to a problem is a combination of well-known techniques and new insights. The techniques that appear in competitive programming also form the basis for the scientific research of algorithms.

¶4 The implementation of algorithms requires good programming skills. In competitive programming, the solutions are graded by testing an implemented algorithm using a set of test cases. Thus, it is not enough that the idea of the algorithm is correct, but the implementation also has to be correct.

¶5 A good coding style in contests is straightforward and concise. Programs should be written quickly, because there is not much time available. Unlike in traditional software engineering, the programs are short (usually at most a few hundred lines of code), and they do not need to be maintained after the contest.

1.1 Programming languages

¶6 At the moment, the most popular programming languages used in contests are C++, Python and Java. For example, in Google Code Jam 2017, among the best 3,000 participants, 79 % used C++, 16 % used Python and 8 % used Java [29]. Some participants also used several languages.

¶7 Many people think that C++ is the best choice for a competitive programmer, and C++ is nearly always available in contest systems. The benefits of using C++ are that it is a very efficient language and its standard library contains a large collection of data structures and algorithms.

¶8 On the other hand, it is good to master several languages and understand their strengths. For example, if large integers are needed in the problem, Python can be a good choice, because it contains built-in operations for calculating with large integers. Still, most problems in programming contests are set so that using a specific programming language is not an unfair advantage.

¶9 All example programs in this book are written in C++, and the standard library's data structures and algorithms are often used. The programs follow the C++11 standard, which can be used in most contests

nowadays. If you cannot program in C++ yet, now is a good time to start learning.

C++ code template

¶10 A typical C++ code template for competitive programming looks like this:

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // solution comes here
}
```

¶11 The #include line at the beginning of the code is a feature of the g++ compiler that allows us to include the entire standard library. Thus, it is not needed to separately include libraries such as iostream, vector and algorithm, but rather they are available automatically.

¶12 The using line declares that the classes and functions of the standard library can be used directly in the code. Without the using line we would have to write, for example, std::cout, but now it suffices to write cout.

¶13 The code can be compiled using the following command:

```
g++ -std=c++11 -O2 -Wall test.cpp -o test
```

¶14 This command produces a binary file test from the source code test.cpp. The compiler follows the C++11 standard (-std=c++11), optimizes the code (-O2) and shows warnings about possible errors (-Wall).

Antti Laaksonen, *Competitive Programmer's Handbook*, 第 1 章 引言及 1.1 节,CC BY-NC-SA 4.0。

§ 8 个关键词

design / implementation 设计 / 实现
efficient 高效的
concise 简洁的
maintained (代码)被维护
participants 参赛者
standard library 标准库
compile / compiler 编译 / 编译器
warnings 警告

§ After you read

Q1 What two topics does competitive programming combine?

Q2 Why is a correct idea not enough in a programming contest?

Q3 Why do many people think C++ is the best choice for contests?

Write one sentence in English to sum up this section:

§ 中文对照(先读完英文再看)

¶1 竞赛编程结合了两个方面:(1)算法的设计,(2)算法的实现。

¶2 算法设计包括解决问题和数学思维,需要分析问题、创造性地解决问题的能力。解决问题的算法必须既正确又高效,而问题的核心往往就在于发明一个高效的算法。

¶3 算法的理论知识对竞赛选手很重要。通常,一道题的解法是一些著名技巧和新想法的组合。竞赛编程里出现的这些技巧,也是算法科学研究的基础。

¶4 实现算法需要良好的编程能力。在竞赛编程中,评判解法的方式是用一组测试数据去测试实现出来的算法。因此,只有算法的思路正确是不够的,实现也必须正确。

¶5 比赛中好的代码风格是直截了当、简洁的。程序要写得快,因为时间不多。和传统的软件工程不同,这些程序很短(通常最多几百行),比赛结束后也不需要维护。

¶6 目前比赛中最流行的编程语言是 C++、Python 和 Java。例如在 2017 年 Google Code Jam 的前 3000 名参赛者中,79% 用 C++,16% 用 Python,8% 用 Java [29]。也有些参赛者用了好几种语言。

¶7 很多人认为 C++ 是竞赛选手的最佳选择,而且比赛系统里几乎总能用 C++。用 C++ 的好处是它非常高效,它的标准库里有大量现成的数据结构和算法。

¶8 另一方面,掌握几种语言、了解它们各自的长处也是好事。例如题目里需要大整数时,Python 可能是个好选择,因为它内置了大整数运算。不过,比赛中的大多数题目都设计得让使用某种特定语言不会带来不公平的优势。

¶9 本书所有示例程序都用 C++ 写,并且经常用到标准库的数据结构和算法。程序遵循 C++11 标准,如今大多数比赛都能用它。如果你还不会用 C++ 编程,现在正是开始学的好时候。

¶10 竞赛编程中一个典型的 C++ 代码模板如下:〔代码见原文〕

¶11 代码开头的 #include 这一行是 g++ 编译器的一个特性,能一次引入整个标准库。因此不需要分别引入 iostream、vector、algorithm 这些库,它们自动就能用。

¶12 using 这一行声明标准库的类和函数可以在代码里直接使用。没有这一行,我们就得写 std::cout 之类,现在只写 cout 就够了。

¶13 代码可以用下面的命令编译:〔代码见原文〕

¶14 这条命令从源文件 test.cpp 生成一个叫 test 的可执行文件。编译器遵循 C++11 标准(-std=c++11),对代码做优化(-O2),并对可能的错误给出警告(-Wall)。

译文据 CC BY-NC-SA 4.0 原文翻译,仅供个人学习。

§ 这一节的代码

C++ 模板做了三件事:一次引入整个标准库、省掉 `std::` 前缀、留一个 main 写解法。Rust 的标准库默认就能用,常用的东西用 `use` 引进来;下面是你在 CSES 里一直在用的对应写法。`-O` 相当于 `-O2`,Rust 默认就会给出警告。

```rust
use std::io::{self, Read};

fn main() {
    let mut input = String::new();
    io::stdin().read_to_string(&mut input).unwrap();
    // solution comes here
}
```

```
rustc -O main.rs -o main
```

能用在哪类 CSES 题上:所有题都用这个模板;“思路对还不够、实现也要对”说的正是 CSES 用测试数据判题。作者提到的大整数,对应你明天 1092 要注意的:Rust 没有内置大整数,先估最大值再选 i64。

路线:今天是 118 段里的第 1 段。上一段:没有,这是起点。下一段:1.2 Input and output,讲比赛里怎么读输入、写输出,怎么让输入输出更快。

Reference answers: ① The design of algorithms and the implementation of algorithms. ② Because solutions are tested with test cases, so the implementation must be correct too. ③ Because C++ is very efficient and its standard library has many data structures and algorithms. · Reference summary: Contest programming needs both a correct idea and a correct implementation, and C++ is the most popular language because it is fast and has a large standard library.

☐ **读书:《1984》第一部 第 4–5 章** —— 接着你读完的第 3 章。留意主角上班时处理的是什么样的文件、这份工作的“规矩”是什么,以及食堂里那位编词典的同事怎么谈论语言。 选做

☐ **运动:无氧 · 徒手力量** —— 热身几分钟,深蹲 3×12、俯卧撑 3×8–10(做不动改跪姿)、臀桥 3×15、平板支撑 3×30 秒,最后拉伸大腿、胸和肩。 选做

☐ 填写今晚的记录 必做

做了一点也写上做到第几步(项目填在记录页最上面的项目一栏);照实选“怎么完成的”,它只用来调提示多少;重写了就勾上;CodeCrafters 关卡填编号(例如 at4,做完 ia4 就填 ia4);选做的做了就勾。备注里想说什么都可以写,明天的简报开头会回你;明天从这里写的位置接着排,打卡系数也按这里算。